



MVC Pattern Summary:

Model-View-Controller

The **Model-View-Controller (MVC)** is an architectural design pattern used to separate an application into three interconnected parts. The primary goal is to separate the application's **business logic** (what the app does) from the **presentation** (how the app looks).

This separation of concerns makes large applications much easier to develop, test, and maintain.

1. The Three Core Components

Component	Role	Java Analogy
Model	Data and Business Logic. Manages the application's data, rules, and state. It is completely independent of the user interface.	Plain Old Java Objects (POJOs), Services, Repository classes that talk to a database.
View	Presentation Layer (UI). Responsible for displaying the data received from the Model to the user and capturing user input (e.g., button clicks).	HTML/JSP, Thymeleaf templates, or Java Swing/FX UI components.
Controller	Traffic Cop / Intermediary. Handles user input, processes requests, determines the required Model changes, and selects the appropriate View to display the result.	Classes annotated as <code>@Controller</code> in Spring or Servlets in traditional Java EE.

2. The Flow of Control (The Request Cycle)

A user interaction follows a specific, cyclical path through the MVC components:

1. **User Interaction:** The user interacts with the **View** (e.g., clicks a "Submit" button).
2. **Request to Controller:** The **View** sends the request (input) to the **Controller**.

3. **Controller Delegation:** The **Controller** receives the input, interprets the action, and delegates the necessary work to the **Model** (e.g., "Save this new user data").
4. **Model Manipulation:** The **Model** updates its internal state or retrieves/saves data (e.g., interacts with the database).
5. **Model Notification:** The **Model** notifies the **View** that its data has changed (or the Controller retrieves the updated data).
6. **View Update:** The **View** receives the updated data from the Model and renders the output back to the user's screen.

3. Benefits of Using MVC

- **Decoupling:** Changes to the UI (View) rarely affect the data logic (Model).
- **Maintainability:** Easier to fix bugs in one layer without breaking others.
- **Testability:** The Model can be tested independently of the user interface.
- **Parallel Development:** Developers can work on the frontend (View) and backend logic (Model) simultaneously.